Labo 15 — Stockage et StatefulSets

Théorie — comment un cluster qui jette ses Pods garde quand même une base de données : des volumes qui survivent, une classe de stockage qui les fabrique, et un contrôleur qui donne aux Pods un nom et un disque qu'ils retrouvent.

Objectifs

- Comprendre la chaîne **PersistentVolume → PersistentVolumeClaim → Pod**, et le rôle de la **StorageClass**.
- Lire les modes d'accès, la politique de récupération, et ce que `local-path` fait sur k3s.
- Distinguer un **Deployment** d'un **StatefulSet** : identité stable, volume par Pod, service *headless*.
- Déployer PostgreSQL en StatefulSet et vérifier que les données survivent à la suppression du Pod.
- Savoir sauvegarder, et savoir quand une base n'a rien à faire dans le cluster.

---

1. Le problème : des Pods jetables, des données qui ne le sont pas

Au labo 06, la règle était : une donnée qui doit survivre n'est pas *dans* le conteneur. Dans le cluster, c'est pire : un Pod est jetable (labo 11), remplacé à chaque rollout (labo 12), et peut atterrir sur **un autre nœud** où le disque local du précédent n'existe pas. Un `emptyDir` (un volume lié à la vie du Pod) ou un `hostPath` (un répertoire du nœud, labo 11) ne répondent pas à ce besoin. Il faut un stockage qui existe **indépendamment** du Pod et du nœud, que le cluster sache rattacher au Pod où qu'il soit.

> **→ RAPPEL DOCKER**
> Le volume nommé du labo 06 (`podman volume create pgdata`) devient un **PersistentVolume** ; le `-v pgdata:/var/lib/postgresql/data` devient une **PersistentVolumeClaim** référencée par le template ; et `podman volume export` devient… la même chose, un `pg_dump` ou une copie du volume. Ce qui change : le volume peut être un disque réseau (EBS, Azure Disk, NFS, Ceph) attaché au nœud par un pilote, et c'est le cluster qui gère le cycle de vie.

2. Trois objets pour un disque

Objet	Qui l'écrit	Ce qu'il dit
StorageClass	l'équipe plateforme (ou la distribution)	« voici comment fabriquer un volume » : le pilote (*provisioner*), ses paramètres, la politique de récupération, le moment de la création
PersistentVolumeClaim (PVC)	**vous**, dans le namespace de l'application	« je veux 500 Mi, en lecture-écriture, de la classe par défaut »
PersistentVolume (PV)	le *provisioner*, automatiquement	« voici un volume de 500 Mi, sur ce disque, lié à cette PVC »

Le Pod ne connaît que la **PVC** (`persistentVolumeClaim: {claimName: demo-data}`). La StorageClass transforme la demande en volume réel : c'est le **provisionnement dynamique**, qui a remplacé

l'époque où un administrateur créait les PV à la main. `kubectl get pvc` montre `Pending` tant qu'aucun volume n'est lié, puis `Bound` avec le nom du PV (`pvc-<uuid>`).

Deux réglages de la StorageClass comptent au quotidien. **`volumeBindingMode: WaitForFirstConsumer`** : le volume n'est créé que lorsqu'un Pod utilise la PVC — ce qui permet de le créer sur le bon nœud ; une PVC seule reste donc `Pending`, et ce n'est pas une erreur. **`reclaimPolicy: Delete`** : supprimer la PVC supprime le PV **et ses données** ; `Retain` garde le PV orphelin, à nettoyer à la main. Les **modes d'accès** disent qui peut monter : `ReadWriteOnce` (un nœud à la fois — un disque), `ReadOnlyMany`, `ReadWriteMany` (plusieurs nœuds — NFS, un système de fichiers réseau).

> **K3S**
> La StorageClass par défaut de k3s, **`local-path`**, crée un simple répertoire sur le nœud (`/var/lib/rancher/k3s/storage/pvc-<uuid>_<namespace>_<pvc>`) et marque le PV avec une **affinité de nœud** : le Pod ne pourra jamais être planifié ailleurs. Sur un nœud unique, c'est invisible ; sur un vrai cluster, c'est exactement ce qu'on ne veut pas pour une base, et on utilise un pilote **CSI** (*Container Storage Interface*) vers un stockage réseau : `gp3` sur EKS, `managed-csi` sur AKS, Longhorn ou Ceph sur des machines nues. Le manifeste de la PVC ne change pas ; la StorageClass, si.

> **→ LINUX**
> Sous la PVC, il y a toujours un **montage** : le kubelet attache le disque (ou le répertoire) au nœud, le formate s'il le faut (`ext4` par défaut), et le monte dans le conteneur comme un bind mount du labo 06 — avec les mêmes questions d'UID : PostgreSQL écrit sous l'UID 70, et `fsGroup` dans le `securityContext` du Pod sert à donner le volume au bon groupe quand le pilote ne le fait pas.

3. Deployment ou StatefulSet

Un Deployment avec une PVC fonctionne pour **une** instance d'une application qui tolère de perdre un Pod et d'en avoir un autre, à condition que les deux ne tournent jamais en même temps sur le même disque. Pour une base de données, deux choses lui manquent :

- **Une identité stable.** Les Pods d'un Deployment s'appellent `api-6c7bb54fc5-2th65` ; ceux d'un **StatefulSet** s'appellent `db-0`, `db-1`, `db-2`, dans l'ordre, et un Pod recréé **reprend son nom**.
- **Un volume par Pod.** Avec `volumeClaimTemplates`, chaque Pod reçoit **sa** PVC, nommée `data-db-0`, `data-db-1`…, qui lui est rattachée à chaque recréation. Un Deployment à trois réplicas partagerait la même PVC (impossible en `ReadWriteOnce`, catastrophique pour une base).

S'y ajoutent un **démarrage et un arrêt ordonnés** (`db-0` avant `db-1`), une mise à jour **un Pod à la fois** en ordre inverse, et le **Service headless** (`clusterIP: None`) : au lieu d'une IP virtuelle qui répartit, le DNS renvoie directement les IP des Pods, et chaque Pod a son nom, `db-0.db.lab15.svc.cluster.local`. C'est ce qui permet à une réplique `db-1` de savoir que son primaire est `db-0`.

> **À RETENIR**
> Supprimer un StatefulSet **ne supprime pas** ses PVC : les données restent, et un nouveau StatefulSet du même nom les retrouve. Réduire à `replicas: 0` non plus. C'est le filet de sécurité symétrique du `podman compose down` du labo 09 — et c'est pourquoi un namespace supprimé emporte les PVC (et, avec `Delete`, les données) : lisez `kubectl get pvc` avant.

> **→ POSTGRESQL**
> L'image officielle veut un `PGDATA` **vide ou déjà initialisé**, et `lost+found` (créé par `ext4` à la racine d'un disque) la fait échouer : d'où le sous-répertoire `PGDATA=/var/lib/postgresql/data/pgdata` dans le StatefulSet. Et un StatefulSet à `replicas: 3` ne fait **pas** trois réplicas PostgreSQL : il fait trois bases indépendantes. La réplication est l'affaire d'un **opérateur** (CloudNativePG, Zalando, Crunchy) qui gère primaire, réplicas, bascule et sauvegardes — c'est lui que l'on déploie en production, pas un StatefulSet nu.

4. Sauvegarder, encore

La règle du labo 06 ne change pas : **logique** avec `kubectl exec db-0 -- pg_dump -U postgres > dump.sql` (cohérent à chaud), **physique** via un *snapshot* du volume (`VolumeSnapshot`, si le pilote CSI le permet — pas `local-path`) ou une copie Pod arrêté. Un opérateur automatise les deux vers un stockage objet. Et la question « où est la donnée ? » a une réponse en deux commandes : `kubectl get pvc` (quelle PVC), `kubectl get pv <nom> -o yaml` (quel disque, quel nœud, quelle politique).

5. En entreprise

- **La base n'est souvent pas dans le cluster** : un service managé (RDS, Cloud SQL, Azure Database) offre sauvegardes, réplication et mises à jour sans qu'on y touche ; l'application y accède par un Service `ExternalName` ou une ConfigMap avec l'URL. C'est le choix par défaut des équipes qui ne veulent pas exploiter PostgreSQL.
- **Dans le cluster, un opérateur**, jamais un StatefulSet écrit à la main, et une StorageClass réseau avec snapshots.
- **Les StatefulSets « faits maison »** restent légitimes pour les environnements de test, les caches, les files d'attente, et pour comprendre ce que l'opérateur fait à votre place.
- **La PVC est la frontière** : l'application demande de l'espace, la plateforme décide d'où il vient.

À retenir

- Pod → PVC → PV ← StorageClass : l'application demande, la classe fabrique, le cluster lie.
- `WaitForFirstConsumer` = une PVC seule reste `Pending` ; `Delete` = supprimer la PVC supprime les données.
- `local-path` chez k3s = un répertoire du nœud avec affinité ; en production, un pilote CSI réseau.
- StatefulSet = noms stables (`db-0`), une PVC par Pod (`data-db-0`), démarrage ordonné, Service headless (`db-0.db`).
- Supprimer le StatefulSet ou le réduire à zéro garde les PVC ; supprimer le namespace les emporte.
- Trois réplicas d'un StatefulSet PostgreSQL = trois bases ; la réplication est l'affaire d'un opérateur.
- Sauvegarde : `pg_dump` par `kubectl exec`, snapshots CSI, ou un opérateur — et souvent, une base managée hors du cluster.

Vocabulaire

PersistentVolume (PV) : un volume réel, objet du cluster. — **PersistentVolumeClaim (PVC)** : une demande de stockage par l'application. — **StorageClass** : recette de fabrication d'un volume (*provisioner*, paramètres, politique). — **provisionnement dynamique** : création du PV à la demande. — `WaitForFirstConsumer` : création du volume au premier Pod. — `reclaimPolicy` : `Delete` ou `Retain` à la suppression de la PVC. — **modes d'accès** : `ReadWriteOnce`, `ReadOnlyMany`, `ReadWriteMany`. — **CSI** : interface des pilotes de stockage. — **StatefulSet** : contrôleur à identité stable et volume par Pod. — `volumeClaimTemplates` : modèle de PVC par Pod. — **Service headless** : `clusterIP: None`, un nom DNS par Pod. — **opérateur** : contrôleur spécialisé qui exploite une application avec état.